Django Product Model Labs

Lab 1: Guided Lab – Product Model
Lab 2: Guided Lab – Strengthen the Product Model

Lab 1: Guided Lab – Product Model

Goal: translate a single Product entity into Django fields, without relationships or migrations logic beyond the basics.

Steps I Followed

1. Created the catalog app and registered it

```
python manage.py startapp catalog
```

I added 'catalog' to INSTALLED_APPS in settings.py.

2. Defined the Product model inside catalog/models.py.

3. Made sku unique and name required

```
sku = models.CharField(max_length=50, unique=True)
name = models.CharField(max_length=200)
```

unique=True stops two products from sharing the same SKU. name is required by default, since Django fields are mandatory unless I say otherwise.

4. Made description optional

```
description = models.TextField(blank=True)
```

blank=True lets the field stay empty in forms/admin, since a description isn't essential data for a product.

5. Added a Category using TextChoices with three options

```
class Category(models.TextChoices):
    BOOKS = 'BK', 'Books'
    ELECTRONICS = 'EL', 'Electronics'
    HOME = 'HM', 'Home & Kitchen'

category = models.CharField(max_length=2, choices=Category.choices)
```

6. Added price and stock with the right field types

```
price = models.DecimalField(max_digits=10, decimal_places=2)
stock = models.PositiveIntegerField()
```

7. Added is_active, created_at, and updated_at

```
is_active = models.BooleanField(default=True)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
```

auto_now_add sets the timestamp once at creation; auto_now updates it every time the record is saved.

8. Added __str__() and ran the system check

```
def __str__(self):
    return self.name
```

I ran python manage.py check and it came back with no issues.

Exit Ticket

Why does price use DecimalField, and why does description use blank=True?

I used DecimalField for price because it stores numbers with exact precision. FloatField uses binary floating-point representation, which can introduce small rounding errors (e.g. 19.990000000000002 instead of 19.99), and that's risky for money-related calculations where even tiny errors add up. I made description blank=True because a description isn't essential information about a product — unlike the name or SKU — so it makes sense to let it stay empty without blocking the save.

How I Verified It

I ran makemigrations and migrate, then went into the Django shell and created a test product to confirm __str__() returned the right name. I also started the dev server and checked that the default Django success page loaded fine in the browser.

Lab 2: Guided Lab – Strengthen the Product Model

Goal: add behaviour, metadata, indexes, and constraints to yesterday's Product model.

Steps I Followed

1. Added is_available() using is_active and stock

```
def is_available(self):
    return self.is_active and self.stock > 0
```

It only returns True when both conditions are true – the product has to be active AND have stock above zero.

2. Added inventory_value() using price and stock

```
def inventory_value(self):
    return self.price * self.stock
```

This calculates the total value of the stock currently held for that product.

3. Set default ordering by category then name

```
class Meta:
    ordering = ['category', 'name']
```

Now every query on Product sorts by category first, then alphabetically by name within each category.

4. Added clear singular and plural verbose names

```
verbose_name = 'Product'
verbose_name_plural = 'Products'
```

5. Added a composite index on category and is_active

```
indexes = [
    models.Index(fields=['category', 'is_active'], name='product_cat_active_idx'),
]
```

This speeds up queries that filter on both fields together, which is a common lookup for an e-commerce catalog.

6 & 7. Added named constraints requiring price and stock to be non-negative

```
constraints = [
    models.CheckConstraint(condition=models.Q(price__gte=0), name='price_non_negative'),
    models.CheckConstraint(condition=models.Q(stock__gte=0), name='stock_non_negative'),
]
```

One thing I ran into: Django 6.1 renamed the CheckConstraint keyword from 'check' to 'condition'. I caught this from a warning and fixed it.

8. Ran the system check and thought through each design choice – covered above and in the exit ticket below.

Exit Ticket

Why does non-negative stock deserve both friendly validation and a database constraint?

PositiveIntegerField gives friendly validation at the earliest point – it stops an invalid value in a form or the admin and gives a clear, immediate error before anything gets saved. The CheckConstraint is the last line of defense at the database level. Even if a script, an API call, an admin action, or a direct shell command skips the form entirely, the database itself will still reject the write. I tested this myself by trying to save a product with a negative price directly from the Django shell, and it raised:

```
django.db.utils.IntegrityError: CHECK constraint failed: price_non_negative
```

That proved to me the constraint enforces data integrity no matter how the data gets written, while the field-level validation is really just there to make the experience smoother for a normal user going through a form.

How I Verified It

I ran makemigrations (which generated a migration recording the new Meta options, index, and constraints) and migrate. Then I tested is_available() and inventory_value() in the shell, and they returned True and 259.90, which confirmed the methods were working correctly against real stored data.